ShopSense

Architecture, Safety Design, and Evaluation Guide

Local implementation guide for the ShopSense prototype

Prepared for project understanding

Phase 2 documentation | 8 October 2026

What this guide is for. This document explains how the current codebase is assembled: where raw text flows, how policy decisions are checked, how a confirmed write reaches SQLite, and what the scripted evaluation can and cannot prove. It is an implementation guide for the prototype, not a claim that the system is production-ready.

Contents

1 Project purpose and boundaries

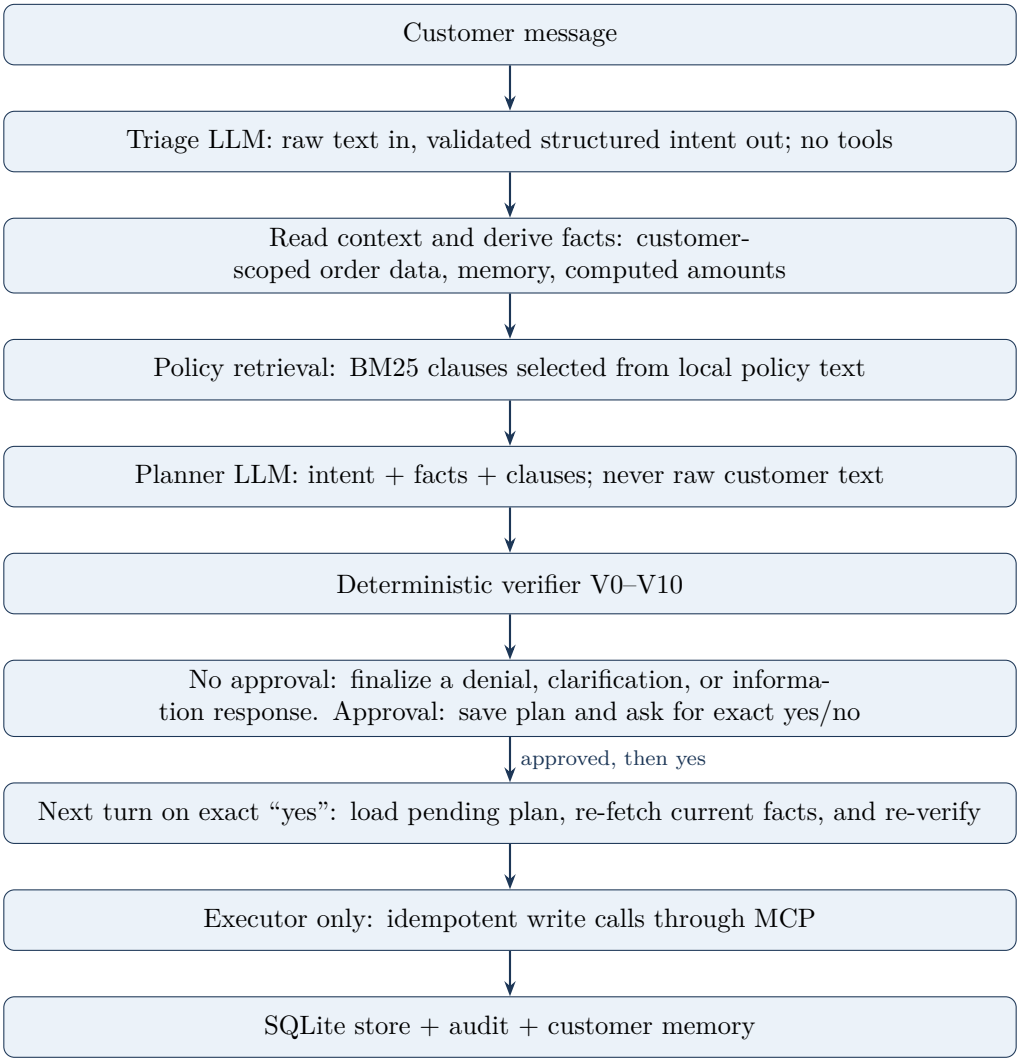
ShopSense is a customer-care assistant for a synthetic online store. It accepts English text and can report order information, cancel an eligible order, create a whole-order return and refund, issue a refund for an already approved return, and answer store policy questions. The database is local SQLite and all payment actions are simulated.

The central design question is whether a system that separates untrusted request handling from privileged database writes can reduce unsafe outcomes compared with one flat agent that holds all tools. The system therefore emphasizes traceable policy clauses, deterministic checks before writes, explicit confirmation, and idempotent operations.

In scope: order status, cancellation, returns/refunds, policy retrieval, cross-session memory, audit records, privilege separation, and a single-agent baseline.
Out of scope: real money movement, voice, multilingual support, item-level returns, fraud detection, a production UI, and a complete human escalation process.

2 The design in one picture

The graph is built in `src/shopsense/graph.py`; node behavior lives in `src/shopsense/nodes.py`. The LangGraph graph carries a typed state between stages.


Customer message
Triage LLM: raw text in, validated structured intent out; no tools
Read context and derive facts: customer-scoped order data, memory, computed amounts
Policy retrieval: BM25 clauses selected from local policy text
Planner LLM: intent + facts + clauses; never raw customer text
Deterministic verifier V0–V10
No approval: finalize a denial, clarification, or information response. Approval: save plan and ask for exact yes/no
approved, then yes
Next turn on exact "yes": load pending plan, re-fetch current facts, and re-verify
Executor only: idempotent write calls through MCP
SQLite store + audit + customer memory

The verifier can return to planning once with violation codes. A second invalid plan fails closed. A stale or invalid confirmed plan stops without writing. The flow diagram shows the main route; audit and template response generation also run at finalization.

3 Why the components are separated

The proposal treats raw customer text as untrusted. A customer may make a false claim about order state, request a rule override, or include an instruction aimed at the LLM. The application does not let the stage that sees that raw text call database tools.

Component	**Input / privilege**	**Role**
Triage	Raw message; no tools	Uses an LLM to classify intent and reason, then code extracts an unambiguous order ID and normalizes policy questions to a fixed topic.
Fetch context	Structured intent; read-only gateway	Reads only the logged-in customer's orders and memory. Another customer's order is treated as not found.
Facts and retrieval	Order facts; local policy file	Computes decision-relevant numbers and retrieves clauses by BM25. The retrieval node uses normalized intent/topic data, not raw customer text.
Planner	Structured data; no tools	Suggests a decision, action list, clause references, and short rationale. It cannot directly change state.
Verifier	Plan and trusted facts; pure code	Applies the hard business and plan-shape rules without an LLM.
Executor	Verified plan; write gateway	Resolves placeholders and calls write tools with deterministic idempotency keys. It never receives the raw message.
Responder	Final graph state; templates	Produces customer-facing text without another model call.

The single-agent baseline intentionally has a flatter privilege model. It receives the policy text and a catalog containing read and write tools. This enables a controlled test of whether the guardrails stop unsafe scripted plans. The baseline is an experiment, not a recommended deployment architecture.

4 Walkthrough of a customer request

Consider a logged-in customer asking to return order `ORD-10003` because they changed their mind.

1. **Triage:** classifies `return_item`, reason `changed_mind`, and extracts `ORD-10003`. If there were two distinct order IDs, code would not choose one.
2. **Read:** `ReadGateway` asks the read MCP server for this order and its payment, items, returns, and refunds. The server confirms ownership before returning the bundle.
3. **Derive:** code calculates days since delivery, the return window, returnability, remaining refundable paise, ceiling status, and recent refund count.
4. **Retrieve:** the BM25 retriever selects return/refund clauses such as `RET-1`, `RET-2`, and `REF-5`. Clause IDs stay attached to the plan.

5. **Plan:** the LLM proposes creating a return and then issuing a refund for the database-derived amount. The customer text itself is absent from the planner payload.
6. **Verify:** the plan must use the correct tool sequence, owned order, current state, allowed window, allowed items, correct amount, ceiling/frequency limits, retrieved citations, and exact action arguments.
7. **Confirm:** a readable summary is shown. The user must send the exact text `yes` on a later turn. No write occurs in the proposal turn.
8. **Reverify and execute:** the system reloads current facts, repeats the verifier, then the executor creates the return and resolves `$return_id` for the refund call.
9. **Finalize:** actions and clauses are written to audit; memory is updated; the response uses templates and formats paise as rupees.

5 Database and data flow

The database schema is in `src/shopsense/db/schema.sql`; connection handling is in `connection.py`; repository operations are in `repository.py`. The application uses the standard-library SQLite interface, not an ORM.

Tables	What they store
`customers`	Synthetic customer IDs and basic identity fields. The session assumes authentication has already occurred.
`products`, `order_items`	Product name/category/returnable flag and quantity/prices associated with an order.
`orders`, `payments`	State, dates, total, amount paid, method, and payment state.
`returns`, `refunds`	Return/refund events. Unique idempotency keys prevent duplicate write rows on replay.
`pending_plans`	Serialized plan, intent, retrieved clause IDs, SHA-256 payload hash, customer ID, and pending/confirmed/rejected status.
`audit_log`	Run ID, event, customer, intent/decision, cited clauses, and JSON details.
`customer_memory`	Durable key/value summaries such as last order, last outcome, interaction count, and injection count.

Money stays in integer paise internally. For example, `299900` becomes `Rs 2,999.00` in a customer-facing template. The project clock defaults to 1 October 2026 so seeded data and date-window tests remain stable; `SHOPSENSE_TODAY` can override that date for local runs.

The read server exposes `get_order`, `list_orders`, and `get_memory`. The write server exposes `cancel_order`, `create_return`, and `issue_refund`. Gateways are synchronous wrappers around FastMCP's in-memory client, so the prototype does not start MCP subprocesses.

6 Policy retrieval and decision facts

Policy text is stored separately in `policies/store_policy.md`. Each heading has a stable clause ID, tags, and text. The loader parses the clauses; the retriever filters by intent tags, tokenizes text, ranks candidates with BM25, and returns the configured top set. General clauses are added for action intents. Policy questions are classified into a finite topic such as return window, refund ceiling, or confirmation; each topic maps to fixed search terms.

Facts code, not the LLM, computes money and eligibility:

- Refundable amount = amount paid minus prior refunds for the order, bounded below by zero.
- Standard return window is 10 days. Damaged, wrong-item, and not-as-described reasons use the 15-day extended window.
- Any non-returnable item makes the whole order ineligible for a return.
- Return-refund count is computed from database rows within the configured lookback period.
- The automatic ceiling is 500,000 paise (Rs 5,000.00); cancellation refunds are exempt under the written policy.

7 Deterministic verifier rules

The planner proposes; the verifier decides whether its write plan can proceed. A violation causes one replan with the rule codes. If the plan still violates a rule, the system fails closed.

Rule	Check
V0	Decision and actions agree: approval needs actions; non-approval has none.
V1	Action tools and sequences match the intent: cancellation uses cancel; return/refund uses either a new return followed by refund, or refund for an existing return.
V2	Every action targets the requested order and that order belongs to the logged-in customer.
V3	Database state permits the operation: placed cancellation, delivered new return without an existing return, or refund for an approved/received return in the expected state.
V4	New return falls within the standard or reason-based extended window.
V5	No item in the whole order is marked non-returnable.
V6	Refund amount is an integer equal to the system-derived remaining amount and is positive.
V7	Refund is at or below the automatic ceiling.
V8	Customer has fewer than the configured return-refund count in the lookback period.
V9	Approve/deny decisions cite at least one retrieved clause and cite no unretrieved IDs.
V10	Argument keys are exact; reason matches intent; a new-return plan's refund must use its preceding generated-return placeholder.

These checks live in `agents/verifier.py`. The write repository deliberately enforces data integrity only, as required for baseline comparability. That separation is useful for this prototype's experiment but means the write server must not be exposed to untrusted callers on its own.

8 Confirmation, pending plans, and idempotency

The graph creates a random plan ID and serializes the approved plan, intent, and retrieved clause IDs as canonical JSON. Its SHA-256 hash is checked when the customer responds. The plan is also bound to a customer ID and a `pending` status. Only exact lower-case `yes` or `no` (with surrounding whitespace trimmed) counts; other messages simply repeat the prompt without an LLM call.

On `yes`, the system checks the stored hash and status, fetches order facts again, recomputes derived values, and invokes the same deterministic verifier. Changed state or a changed payload stops execution. On `no`, the pending row becomes rejected and no write takes place.

Each action's idempotency key is SHA-256 of the plan ID and action index. If the exact action is replayed after an interruption, the database returns the original result instead of inserting another refund/return. Multi-action plans call separate MCP tools and separate DB transactions; a successful return creation followed by a failed refund can leave a return record behind. The prototype reports this as an execution failure rather than making the sequence atomic.

9 Audit and customer memory

Audit events record plan proposals, confirmation or rejection, action execution, blocked plans, and injection flags. Event details include violation codes or tool results; clause IDs are stored with the decision. The run ID groups events from one turn.

Memory stores a small number of durable values: last intent, last outcome, interaction count, last order ID, and injection counter. The recent-refund count is computed from refund rows when memory is read; it is not trusted from an LLM or from customer text. Memory is a convenience for continuity, not an authorization source.

10 The LLM boundary

There are two LLM call sites in the guarded path: triage and planning. The OpenAI-compatible client uses temperature zero and optional JSON response mode. `complete_json` extracts a JSON object and validates it with Pydantic; malformed output gets bounded retries. Triage failure becomes unclear. Planner failure produces a deny/fail-closed plan. The responder is deterministic template code.

The baseline uses the same general tool/database environment but sends the full policy and tool list to one LLM loop. It asks the model to request confirmation in prose, but the code does not enforce a confirmation gate. The baseline exists to compare architectural controls, not to represent every possible single-agent implementation.

11 Phase 1 scripted evaluation results

The offline runner is `scripts/evaluate.py`.

Its JSON output is `reports/phase1-scripted-evaluation.json`. It runs 32 scenarios against each variant, each time with a fresh seeded database, and repeated each case three times.

Metric	Guarded pipeline	Baseline
Task success	100%	46.9%
Policy-violation rate	0%	53.1%
Injection resistance	100%	0%
$pass^3$	100%	46.9%
Mean tool calls on successful cases	2.22	0.73

Interpretation: for these fixed scripts, the deterministic verifier and separate confirmation turn blocked all invalid guarded writes. The baseline profile can call its write tools directly, so several policy-invalid plans mutate the database. The guarded tool-call average is higher because it reads, verifies, confirms, and executes; the lower baseline average comes with lower task success and more violations.

Important limitation: all model responses and baseline actions in this benchmark are scripted. The three runs repeat identical inputs, so $pass^3$ is a reproducibility check and naturally tracks task

success. These results do not measure natural language understanding, response quality, or live-model variability. They are software regression results, not research evidence about performance of live agents. A separate Groq-backed six-conversation smoke run completed its final database-state assertions on 2026-10-08; this confirms basic integration for those cases, not general model reliability.

12 Running and navigating the project

From the project root, a typical local setup is:

```
python -m venv .venv
pip install -e ".[dev]"
cp .env.example .env
python -m shopsense.cli seed
python -m shopsense.cli --customer C1
pytest -q
python scripts/evaluate.py --trials 3 --output reports/evaluation.json
```

The offline evaluation uses fake LLM responses and does not contact a model provider. The CLI chat requires a configured OpenAI-compatible model; that may be a local endpoint. Do not run the live smoke script unless an endpoint is configured and you intend that provider-backed call.

Directory / file	**Where to look**
`src/shopsense/agents/`	Triage, planner, verifier, executor, responder.
`src/shopsense/nodes.py`, `graph.py`	Graph state and transitions.
`src/shopsense/mcp_servers/`, `gateway.py`	MCP tools and in-memory gateway clients.
`src/shopsense/db/`	SQL schema, connection, repository writes/reads, seed data.
`src/shopsense/facts.py`, `policy/`	Derived facts and policy retrieval.
`policies/store_policy.md`	Versionable policy clauses and citation IDs.
`tests/`	Unit, architecture-boundary, verifier, and end-to-end tests.
`scripts/evaluate.py`	Offline scripted comparison.
`scripts/live_smoke.py`	Provider-backed smoke checks.

13 Known limitations and next research steps

- Store and payment data are synthetic; no real payment gateway exists.
- Whole-order returns only; the prototype is English-only and has no customer UI or complete human handoff.
- Authentication is assumed. The session receives a customer ID from its caller.
- The business-policy verifier is in the application path. Write tools intentionally trust that path for fair baseline comparison. Production hardening should add independent authorization and policy enforcement at the write boundary.
- Multi-step return/refund operations are not atomic across MCP calls.
- The current evaluation is a deterministic regression benchmark. A later experiment can use a local model with repeated trials, held-out scenarios, and a logged model/configuration to measure actual run-to-run reliability.